

Chengxi Yang

✉ arcadia-y@sjtu.edu.cn / arcadia-y.github.io

🎓 EDUCATION

Shanghai Jiao Tong University

Bachelor of Computer Science

Shanghai, China

Sep. 2022 - Jun. 2026 (expected)

- A member of ACM Honors Class, an elite CS program in SJTU
- **GPA: 4.0/4.3**, Ranking: 3/30, 22 A+ courses
- Selected Courses: Mathematical Logic: 100/100, Programming Verification: 99/100, Model Checking: 100/100, Compiler Design and Implementation: 97/100, Cryptography: 100/100, Reinforcement Learning: 100/100

❓ RESEARCH INTEREST

My research interest mainly lies in **programming languages**, especially **formal verification**, aiming to make verification more practical and efficient. I'm also broadly interested in other areas related to PL, such as program analysis, compiler design and AI4FTP (AI for Formal Theorem Proving).

🔍 RESEARCH EXPERIENCE

University of Wisconsin-Madison

Research Intern at madPL, co-advised by Prof. Thomas Reps and Prof. Tej Chajed

Madison, WI, USA

Jun. 2025 - present

Resource Analysis for DBSP (Ongoing)

- Our primary goal was to build a resource analysis framework in Lean for DBSP, a simple but expressive language for stream computations with a general incremental algorithm, but we also found some other interesting results.
- I formalized the denotational semantics and a corresponding cost model for DBSP circuits, developing Hoare-style logics for functional correctness and resource analysis using step-indexing.
- I identified the undecidable Fixedpoint Checking (FPC) Problem as a fundamental issue in any DBSP's computation model. I proposed a decidable variant (State Fixedpoint) with a sound and complete checking algorithm, which can be generalized to arbitrary time dimensions along with other DBSP concepts.
- I'm applying the framework to analyze and compare the resource costs of both unoptimized and optimized incremental circuits for two key examples from the DBSP paper: the SQL join and the graph transitive closure.

Shanghai Jiao Tong University

Research Assistant, advised by Prof. Qinxiang Cao

Shanghai, China

Jun. 2024 - Jun. 2025

Formalizing Temporal Reasoning Using the Trace Method (Ongoing)

- We observe that natural proofs for some algorithms involve temporal reasoning about program history, which is hard to formalize with standard Hoare logic. Examples include textbook proofs for DFS and BFS.
- To formalize such temporal reasoning, I proposed a trace method, which embeds the program trace as a ghost state for reasoning. I also introduced novel proof rules that faithfully reflect common reasoning patterns about program history, including rules for both recursions and loops.
- We have formalized the DFS and the Tarjan algorithm using the trace method, and I formalized the DFS. We're still working on the BFS example and exploring how generally this method could be applied.

A Formal Framework for Naturally Specifying and Verifying Algorithms

- We introduce a formal framework in Rocq to specify and verify sequential algorithms more naturally.
- I built the verification framework, including a Hoare logic, program equivalence lemmas and convenient tactics.
- I proposed a novel two-stage proof approach to improve verifications based on the standard Hoare logic. It separates natural logical reasoning from mechanical composition, better mirroring the logical structure of natural proofs.

Contributing to the C Verification Tool QCP

- I designed some new features of our C verification tool QCP, including “multi-branch” which enables fine-grained controls of branches in symbolic execution and “multi-loop” as its loop case.
- I designed and implemented the VS-Code extension for our tool. I'm also the maintainer of the extension.
- I have verified several examples using the tool and contributed to its backend Rocq library.

PUBLICATIONS

A Formal Framework for Naturally Specifying and Verifying Sequential Algorithms

*Chengxi Yang**, *Shushu Wu**, *Qinxiang Cao*

In Theoretical Aspects of Software Engineering (TASE) 2025. [\[arxiv\]](#) [\[code\]](#)

QCP: A Practical Separation Logic-based C Program Verification Tool

Xiwei Wu, *Yueyang Feng**, *Xiaoyang Lu**, *Tianchuan Lin*, *Kan Liu*, *Zhiyi Wang*, *Shushu Wu*, *Lihan Xie*,

Chengxi Yang, *Hongyi Zhong*, *Naijun Zhan*, *Zhenjiang Hu*, *Qinxiang Cao*

In submission. [\[arxiv\]](#)

SELECTED PROJECTS

- **Mx* Compiler:** A Compiler from Mx* language (a C & Java like language) to RISC-V RV32I Assembly implemented in Java, featuring linear scan register allocation, SSA transformation and various optimization.
- **LTL Model Checker:** A Linear Temporal Logic (LTL) model checker implemented in Rust. Given a transition system (TS) and an LTL formula, it can check whether the TS or its arbitrary state satisfies the formula.
- **RISC-V CPU:** A Tomasulo-based RISC-V RV32I CPU designed in Verilog RTL, featuring instruction cache and branch predictor. It can be run on FPGA board.
- **Acore:** A toy RISC-V kernel implemented in Rust that can be run on QEMU. It features remote-procedure-call mechanism, virtual memory and preemptive multi-process scheduling.
- **Query2Label Music Tagging:** I leveraged the Query2Label Transformer framework to improve the existing CNN-based music tagging models.

HONORS & AWARDS

Scholarship

- **Fan Xuji Scholarship** (about 10 winners each year in SJTU), 2024
- **Zhiyuan Honorary Scholarship** (awarded to top 2% students in SJTU annually), 2022-2025

TEACHING

- **TA for Programming(C++).** I gave a lecture on introduction to computer systems, designed a new [course project](#) and a problem in the final exam.
- **TA for Principles and Practice of Computer Algorithms.** I designed a new [lab project](#) on formal verification in Coq.
- **TA for Mathematical Logic.** I answered students' questions, graded their homework and wrote sample solutions for them.
- **TA for Programming Verification.** I maintained the [course git repository](#) and participated in the design of the final project.

SKILLS

Natural Languages: Chinese (native), English (fluent, TOEFL 108).

Programming Languages: Rocq (Coq), Lean 4, Rust, C/C++, Java, Python, Verilog, Go.